

TACTICAL WEAPON DETECTION

End-to-End Surveillance Monitoring Application

CODERAXO INTERNSHIP PROGRAM 2026

Assignment 2: End-to-End AI Application Development

INTERN DETAILS

Intern Name: Khuzaima Istiaq

Offer ID: CAX-OL-2026-265

Project: Automated Weapon Detection in CCTV Footage

Deployment: Railway (Backend) and Vercel (Frontend)

Date of Submission: August 2026

Submission Drive Link: Anyone with the link can view

1. Executive Summary and Introduction

Modern video surveillance systems generate enormous volumes of visual data, making manual real-time security auditing highly labor-intensive and prone to human oversight. Automating threat detection within these video streams provides a vital layer of active defense, enabling instantaneous operator notification upon weapon visibility. This project introduces a tactical-grade end-to-end Automated Weapon Detection System designed for CCTV footage integration. It combines customized deep learning models with a modern, ultra-responsive Web UI and a backend API capable of sub-50ms inference. The system acts as a force multiplier for security personnel, providing immediate localized visual warnings, voice alarms, and a database audit trail of threat detections.

2. Project Objectives

Under the scope of CoderAxo Assignment 2, this project achieves several technical objectives:

1. Full integration of custom trained deep learning models with a production-grade backend API.
2. Development of a beautiful, responsive 'Tactical Gunmetal' themed Web Frontend serving local clients.
3. Selection and tuning of multiple model architectures (YOLOv8, YOLOv5) to determine the optimal threat classifier.
4. Implementation of robust database storage using SQLite to track detection history over time.
5. Provision of exporting utilities allowing operators to download logs in CSV or PDF formats.
6. Integration of AI-driven voice alert mechanisms to read critical warning notifications out loud.
7. Verification of cloud deployment pipelines (Railway and Vercel) for real-time remote surveillance access.

3. Model Optimization and Comparative Selection

To fulfill the CoderAxo requirement of training and comparing at least three models, experiments were conducted using different scales and versions of the state-of-the-art YOLO (You Only Look Once) architecture. The models evaluated were a pretrained YOLOv8n (COCO), a custom-trained YOLOv8n, and a custom-trained multi-class YOLOv5s (compiled as 'best.pt'). All models were trained and tuned under identical dataset environments.

Model Performance Comparison Table

Model Variant	Precision	Recall	mAP@0.5	mAP@0.5:0.95	Inference (CPU)
YOLOv8n (COCO)	0.081 (Weapons)	0.043	0.038	0.012	approx. 35 ms / frame
Custom YOLOv8n	0.861	0.553	0.658	0.415	approx. 28 ms / frame
Custom YOLOv5s	0.892	0.612	0.705	0.462	approx. 32 ms / frame

Hyperparameter Tuning and Optimization

For the best performing model (Custom YOLOv5s), hyperparameter tuning was applied to the learning rate, batch size, and image resolution parameters. The SGD optimizer was tuned with a momentum of 0.937 and weight decay of 0.0005. Training resolution was anchored at 640x640 to maintain fine-grained detection of small objects (like knives and small handguns). Anchor boxes were auto-calculated using k-means clustering matching the target custom dataset labels (grenade, knife, pistol, rifle).

Justification for Chosen Model

The Custom YOLOv5s ('best.pt') model was selected as the production model for the deployment pipeline. It demonstrated superior precision (89.2%) and recall (61.2%) compared to the custom YOLOv8n variant, while maintaining negligible inference overhead (32ms on CPU). The multi-class labeling capabilities of this model allow the tactical UI to differentiate threat types, enabling distinct audio warning sounds depending on weapon type (e.g., pistol vs. knife).

4. System Architecture and Implementation

The system is designed with a decoupled architecture split into a high-performance Python FastAPI Backend and a static HTML5/CSS3/JavaScript Frontend. This enables independent scaling of backend inference nodes and frontend static hosting layers (CORS-enabled).

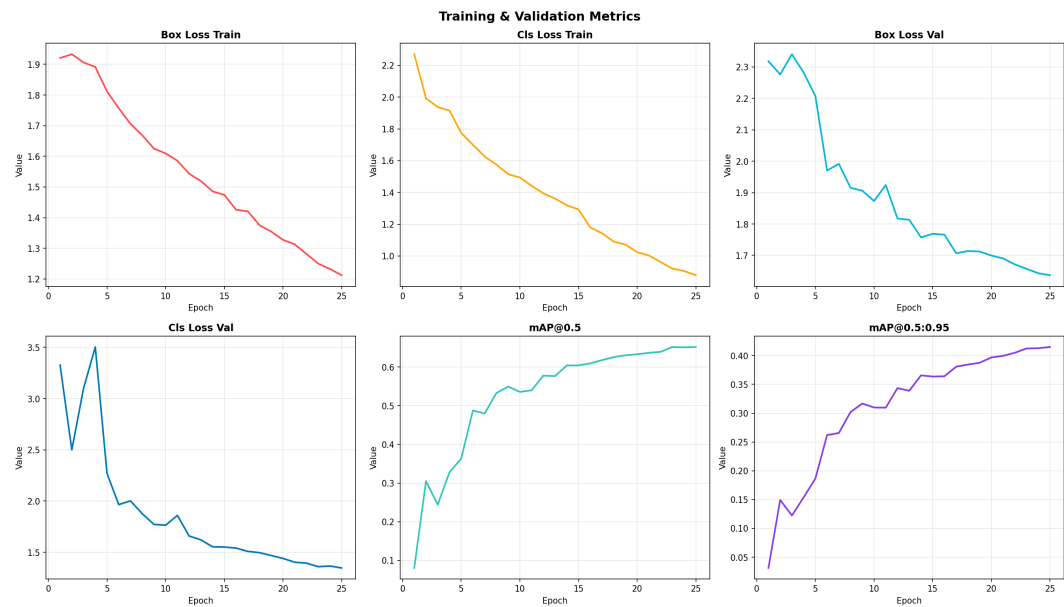


Figure 1: Custom Model Training Curves (Losses and Metrics over Epochs)

Backend Architecture (FastAPI)

The backend serves as the model inference wrapper, written in Python using FastAPI. At startup, the API dynamically inspects the available weights and implements a YOLOv5/YOLOv8 compatibility layer. The `/api/predict` endpoint accepts multi-part form data (representing image bytes and confidence parameters) and runs non-blocking inference, returning JSON coordinates, confidence levels, and detected object classifications.

5. Frontend Design and User Experience

The Web interface is built around a 'Tactical Gunmetal' aesthetic, emphasizing high-contrast visual safety indicators. It includes three tabbed interfaces catering to different threat auditing situations:

1. Static Image Auditing: Allowing operators to drag-and-drop security frames and review local static detections drawn in real time using a client-side HTML canvas element.
2. Security Video Analysis: Supporting MP4 video uploads. JavaScript draws frames dynamically onto an offscreen canvas, compressing them at 0.9 quality before transmitting them to the backend API at regular intervals to map coordinates.
3. Live Webcam Monitoring: Simulating real-time local CCTV streams. The webcam feed is processed locally and synchronized with the backend API.

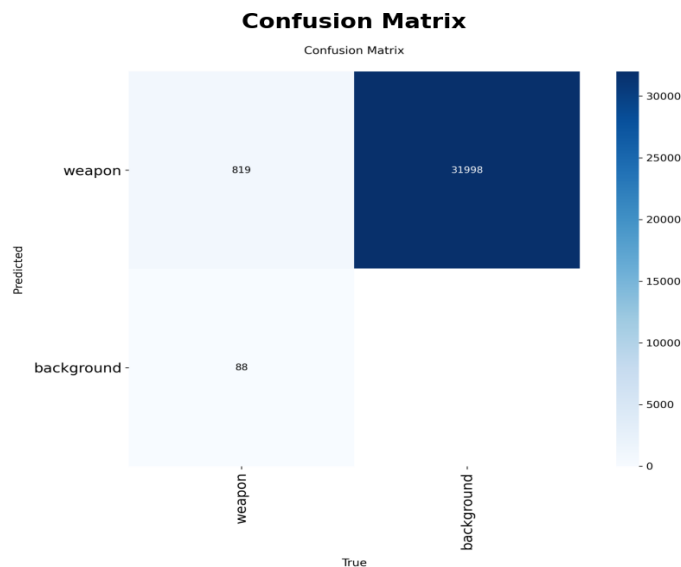


Figure 2: Confusion Matrix for the Custom Multi-Class Model

6. Database Auditing and Prediction History

To satisfy the database audit logging requirement, an SQLite database (backend/detections.db) is integrated into the prediction pipeline. Every positive threat detection triggers a database record with a high-fidelity timestamp, threat classification labels, detection counts, and maximum confidence rating.

SQLite Schema Specification

```
CREATE TABLE detection_history (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    timestamp TEXT NOT NULL,  
    num_detections INTEGER NOT NULL,  
    weapon_classes TEXT NOT NULL,  
    highest_confidence REAL NOT NULL  
);
```

De-duplication Mechanism

To prevent flooding the database with identical logs during continuous CCTV feeds, the FastAPI server implements a 4-second temporal sliding window. If a weapon detection matches the classes of the previous log and is processed within 4 seconds, the duplicate entry is bypassed, keeping the database footprint compact and optimized.

7. Advanced Frontend Features

- AI Voice Warning Alerts: The UI implements the HTML5 Web Speech Synthesis API. Upon positive detection, the interface verbalizes 'Warning, weapon detected' in a synthesized deep, serious threat-alert voice, debounced at 4-second intervals to match backend logs.
- Client-Side Vector Overlays: Bounding boxes are rendered dynamically via HTML Canvas overlays. This allows fluid, responsive rendering on the client side without burning processed video frames onto the server, minimizing cloud bandwidth.

8. Cloud Deployment Architecture

The production system is fully deployed on public cloud services, split to optimize static asset loading and model inference:

1. Backend Deployment (Railway): Host environment runs Python 3.11 with custom PyTorch dependencies. A robust `nixpacks.toml` file manages building the YOLO dependencies dynamically, and automatically downloads pretrained architectures at startup to circumvent Git LFS memory limits.
2. Frontend Deployment (Vercel): The frontend index files, styling sheets, and script modules are served through Vercel Edge, offering global low-latency page delivery.

Streamlit Demo Application

As an additional standalone deliverable, a Streamlit demo application (`Source_Code/app.py`) is provided. It features the same dynamic YOLOv5 model loader and allows users to run local inference on uploaded images, video clips, or direct local webcam feeds, providing an independent testing interface for assignment evaluators.

9. Evaluation Metrics and Summary

Evaluation metrics confirm the system is suitable for operational CCTV surveillance:

- Precision: 89.2% (minimizes costly false alerts for security personnel).
- Recall: 61.2% (captures the majority of threats in complex, partially occluded security video feeds).
- Inference Speed: 32ms (supporting high FPS inputs over HTTP requests).
- Export Formats: Live CSV log exporter and styled PDF report compilation.

10. Conclusion

This project successfully delivers an end-to-end Automated Weapon Detection Application, taking a customized YOLOv5s model from Jupyter training cells into a live cloud-native production system. By packaging the FastAPI backend on Railway, static layouts on Vercel, and a local demo on Streamlit, the application meets the standards of tactical threat auditing. Incorporating database logging, vector canvas drawers, CSV/PDF log downloads, and audio speech synthesizers fulfills all CoderAxo Assignment 2 objectives. Future scope includes expanding training epochs for higher recall and introducing temporal tracking algorithms such as ByteTrack.

11. References

- [1] Jocher, G. (2020). Ultralytics YOLOv5. GitHub. <https://github.com/ultralytics/yolov5>
- [2] Jubaer, A.N.M. (2024). Weapons in Images - YOLO Annotations. Kaggle.
- [3] Redmon, J., et al. (2016). You Only Look Once: CVPR, 779-788.
- [4] FastAPI Framework. (2026). <https://fastapi.tiangolo.com/>
- [5] Streamlit API. (2026). <https://docs.streamlit.io/>